Étude et évaluation d'une architecture Big Data pour le traitement et l'analyse en temps réel des données GPS de véhicules

Oct 28, 2026 · @Wassim May

1. Contexte et objectifs

Le projet construit une plateforme Big Data qui suit en temps réel les taxis et les clients d'un service de VTC, sur le modèle d'Uber ou Bolt, dans le Grand Tunis. Les données sont produites par un simulateur, ce qui permet de tester toute la chaîne sans application mobile réelle.

Objectifs :

- Ingérer en continu les positions GPS des taxis et les demandes des clients.
- Traiter ces flux en temps réel pour mesurer l'offre et la demande par zone.
- Stocker les données pour la recherche et l'historique.
- Visualiser l'activité sur une carte et un tableau de bord rafraîchis toutes les 5 secondes.

Cadre : projet académique Big Data, 3e cycle.

2. Périmètre et besoins fonctionnels

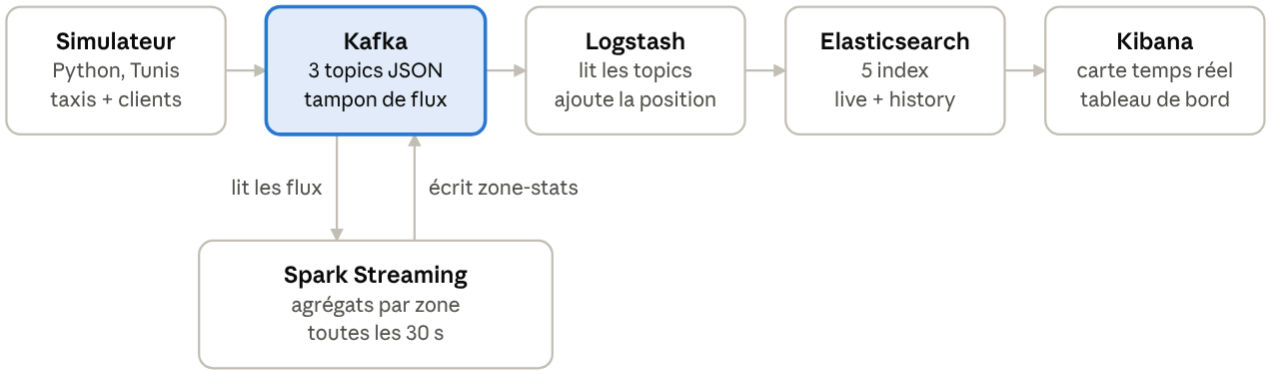
Le périmètre couvre 6 zones du Grand Tunis : Centre-ville, Lac, La Marsa, Ariana, Bardo et Ben Arous. Le paiement, l'inscription des utilisateurs et l'application mobile sont hors périmètre.

Réf.	Besoin	Description
BF1	Simulation	Générer des taxis (par défaut 40) qui se déplacent et des clients qui demandent des courses.
BF2	Attribution	Assigner chaque demande au taxi libre le plus proche (distance de Haversine).
BF3	Cycle de course	Suivre le taxi (libre, vers client, en course) et le client (en attente, assigné, pris en charge, terminé, annulé).

Réf.	Besoin	Description
BF4	Annulation	Annuler une demande sans taxi après 90 secondes.
BF5	Statistiques par zone	Calculer toutes les 30 s les taxis libres et occupés, les clients en attente et les courses terminées ou annulées.
BF6	Carte temps réel	Afficher la dernière position de chaque taxi et les clients en attente.
BF7	Tableau de bord	Montrer les zones les plus demandées, la répartition des statuts et l'activité dans le temps.
BF8	Historique	Conserver toutes les positions et demandes pour l'analyse.

3. Architecture et stack technique

Kafka relie le simulateur au stockage et au calcul temps réel



architecture du pipeline · 6 composants

Le simulateur publie dans Kafka. Logstash copie les flux vers Elasticsearch, que Kibana affiche. En parallèle, Spark lit Kafka, calcule les statistiques par zone et les réécrit dans le topic zone-stats.

Composant	Version	Rôle	Accès local
Kafka + Zookeeper	7.6.1 (Confluent)	Bus de messages temps réel	localhost:9092

Composant	Version	Rôle	Accès local
Kafka UI	latest	Suivi des topics et messages	localhost:8085
Logstash	8.15.0	Ingestion Kafka vers Elasticsearch	localhost:5044
Elasticsearch	8.15.0	Stockage et recherche géographique	localhost:9200
Kibana	8.15.0	Carte et tableau de bord	localhost:5601
PySpark (Jupyter)	latest	Traitement Structured Streaming	localhost:8888, Spark UI 4040
Python + kafka-python	3.12	Simulateur	—
Docker Compose	—	Déploiement de toute la stack	—

4. Données et flux

Les événements circulent en JSON dans 3 topics Kafka, puis sont rangés dans 5 index Elasticsearch.

Topic Kafka	Producteur	Champs principaux
taxi-locations	Simulateur, 1 message par taxi et par seconde	taxi_id, driver, lat, lon, status, zone, speed_kmh, timestamp
client-requests	Simulateur, à chaque changement d'état	request_id, client_id, pickup, dropoff, status, zone, trip_km, timestamp
zone-stats	Spark Streaming, toutes les 30 s	zone, taxis_available, taxis_busy, clients_waiting, trips_completed, trips_cancelled, avg_trip_km

Index Elasticsearch	Contenu	Usage
taxi-locations-live	Dernière position de chaque taxi (1 document par taxi)	Carte temps réel

Index Elasticsearch	Contenu	Usage
client-requests-live	Dernier état de chaque demande	Clients en attente sur la carte
taxi-locations-history	Toutes les positions	Activité dans le temps
client-requests-history	Toutes les demandes	Zones les plus demandées, carte de chaleur
zone-stats	Agrégats Spark	Offre et demande par zone

Les champs location, pickup_location et dropoff_location sont typés geo_point, ce qui permet l'affichage cartographique dans Kibana.

5. Exigences non fonctionnelles

Exigence	Cible
Latence	Une position doit apparaître dans Kibana moins de 10 s après son envoi.
Débit	Au moins 40 positions par seconde, soit 40 taxis, sans perte.
Montée en charge	Le nombre de taxis et le taux de demandes se règlent par paramètre (--taxis, --spawn).
Déploiement	Toute la stack démarre avec une commande : docker compose up -d.
Ressources	Fonctionne sur un PC portable de 16 Go de RAM. Elasticsearch est limité à 1 Go et Logstash à 512 Mo.
Contrôle	Un script (check_pipeline.sh) vérifie chaque étape du pipeline.
Versionnement	Le code est hébergé sur GitHub, avec un README de lancement.

La sécurité d'Elasticsearch est désactivée : l'environnement est local et destiné au développement. Une mise en production demanderait d'activer l'authentification et le chiffrement.

6. Livrables, planning et validation

Livrables :

Livrable	Fichier
Stack Docker	docker-compose.yml
Simulateur	simulator/simulator.py
Pipeline d'ingestion	logstash/pipeline/logstash.conf
Mapping géographique	elasticsearch/setup_index.sh
Traitement temps réel	notebooks/streaming_stats.py
Script de vérification	check_pipeline.sh
Carte et tableau de bord	Kibana : « Carte temps réel » et « Uber Tunis – Temps réel »
Documentation	README.md et ce cahier des charges

Planning, par étapes successives :

1. Mise en place de la stack Docker.
2. Développement du simulateur et envoi vers Kafka.
3. Ingestion Logstash et mapping Elasticsearch.
4. Traitement Spark Streaming par zone.
5. Carte et tableau de bord Kibana.
6. Tests, captures et documentation.

Critères de validation :

- [ ] Les 7 conteneurs sont démarrés.
- [ ] Les topics taxi-locations et client-requests reçoivent des messages en continu.
- [ ] Le nombre de documents augmente dans taxi-locations-history.
- [ ] Le champ location est bien de type geo_point.
- [ ] L'index zone-stats reçoit les agrégats de Spark.
- [ ] Les taxis bougent sur la carte Kibana toutes les 5 s.
- [ ] Le tableau de bord affiche les zones les plus demandées.